Git for Cloud & DevOps

Top 1% Premium Master Documentation
Beginner to Professional GitHub Workflow

Complete Git Lifecycle



This new PDF combines your two uploaded Git PDFs and adds deeper explanations, scenario tasks, wrong vs correct usage, common mistakes, secure repository hygiene, interview preparation, and step-by-step DevOps workflow.

Git Basics | GitHub | Branches | PRs | Merge Conflicts | DevOps

Table of Contents

1. How to use this guide
2. Core mental model: working directory to remote
3. Local Git foundation: init, echo, status, diff, add, commit, delete
4. GitHub connection: clone, origin, SSH, PAT
5. Push, pull, fetch - what, why, when, wrong usage
6. Branches, merge, pull request, GitFlow
7. Undo and recovery: restore, reset, revert, stash, reflog
8. Secure repository hygiene and .gitignore
9. Linux, PowerShell, MobaXterm and Windows notes
10. Scenario-based hands-on labs
11. Common mistakes, errors and fixes
12. Interview questions and scenario answers
13. 7-day practice plan and final cheat sheet

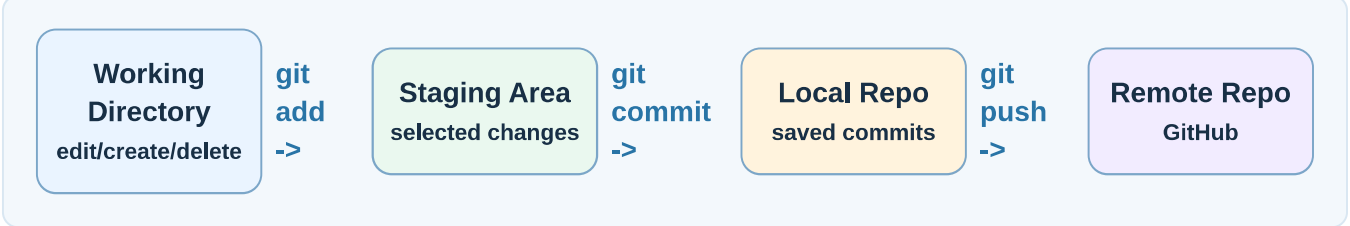
1. How to Use This Guide

Do not memorize Git as random commands. Learn Git as the movement of changes. Every file change starts in the working directory, selected changes go to staging, committed changes become local history, and pushed commits become remote collaboration history.

Learning order	Goal
First	Understand local lifecycle: edit -> stage -> commit
Second	Understand GitHub: remote, push, pull, fetch
Third	Understand team safety: branches and pull requests
Fourth	Learn mistakes and recovery commands
Fifth	Practice scenarios until commands feel natural

> **Professional rule:** review before you stage, review again before you commit, and never push secrets.

2. Core Mental Model: Git Lifecycle



Working Directory means the files you are currently editing. Staging Area means selected changes prepared for the next commit. Local Repository means saved commits on your machine. Remote Repository means GitHub, GitLab, Azure Repos, or another shared server.

Area	Simple meaning	Typical commands
Working Directory	Current files and edits	git status, git diff, git restore file
Staging Area	Prepared changes	git add, git restore --staged, git diff --staged
Local Repository	Saved commits	git commit, git log, git show
Remote Repository	Shared copy	git remote, git push, git pull, git fetch

> **Interview line:** Git does not directly push every edit to GitHub. You stage changes, commit locally, then push commits to the remote repository.

3. Local Git Foundation

3.1 git init

What: `git init` turns a normal folder into a Git repository by creating a hidden `.git` folder.

Why: That folder stores Git tracking data, commit history, branch information, and configuration.

```
mkdir git-practice
cd git-practice
git init
git status
```

Wrong thinking	Correct understanding
git init uploads files to GitHub	No. It only creates a local Git repository.
git init must be run again and again	No. Run it once inside a project folder.
Deleting .git is safe	No. It removes local Git history and tracking.

3.2 echo command for practice

What: `echo` prints text to terminal. With > or >>, it writes text into files.

```
echo "Hello Git" > notes.txt
echo "Second line" >> notes.txt
cat notes.txt
```

Symbol	Meaning
>	Create or overwrite a file
>>	Append/add to existing file

> **Common mistake:** `echo"hi"` is wrong because there must be a space after echo. Correct: `echo "hi" > notes.txt`

3.3 git status, git diff, git add, git commit

`git status` tells you the current state. `git diff` shows what changed before staging. `git add` moves selected changes to staging. `git commit` saves staged changes as a version in the current branch history.

```
git status
git diff
git add notes.txt
git diff --staged
git commit -m "Add notes file"
git show --stat
```

Need	Command
See current state	git status
See unstaged changes	git diff
Stage one file	git add notes.txt
Stage everything	git add .
Review staged changes	git diff --staged
Save checkpoint	git commit -m "message"
Review latest work after commit	git show

> **Key rule:** git commit only saves what is staged. If a file is not staged, it will not be included in that commit.

3.4 Deleting files

Deleting a file is also a change. Git must record that deletion. You can delete manually and then stage the deletion, or use `git rm` to delete and stage in one step.

```
# Manual way
rm notes.txt          # Linux / Git Bash
del notes.txt         # PowerShell
git add notes.txt
git commit -m "Remove notes file"

# Professional shortcut
git rm notes.txt
git commit -m "Remove notes file"
```

Command	Meaning
rm file	Deletes from file system only
git add deleted-file	Stages the deletion

git rm file	Deletes and stages deletion together
delete mode 100644	Git message saying deletion is staged

4. GitHub Connection: Remote, Clone, SSH and PAT

4.1 clone vs init

Situation	Command
New local project first	git init
Existing GitHub repo first	git clone URL
Want team repo locally	git clone URL
Connect local repo to GitHub	git remote add origin URL

4.2 remote origin

A remote is a saved connection between your local repository and GitHub. The default remote name is usually `origin`. origin is not a branch; it is the remote name. `origin/main` means the main branch on the origin remote.

```
git remote add origin https://github.com/USERNAME/REPO.git
git remote -v
git remote set-url origin git@github.com:USERNAME/REPO.git
```

Mistake	Why wrong	Solution
Running remote add origin repeatedly	origin already exists	Use git remote -v or git remote set-url origin URL
Thinking origin means main	origin is remote name	Use origin/main for remote main branch
Wrong repo URL	Push goes to wrong repo or fails	Copy correct HTTPS/SSH URL from GitHub

4.3 SSH vs HTTPS/PAT

For Linux VM, MobaXterm, and DevOps usage, SSH is commonly preferred. For Windows HTTPS usage, GitHub uses Personal Access Tokens (PAT) instead of account passwords for command-line Git authentication.

Use case	Recommended authentication
Linux VM / MobaXterm / Ubuntu	SSH key
Windows PowerShell beginner	HTTPS + PAT or Git Credential Manager
Automation / long-term integration	Service connection, deploy key, or platform token
Public README or docs	Never include private key, PAT, or passwords

```
# SSH key example
ssh-keygen -t ed25519 -C "your_email@example.com"
cat ~/.ssh/id_ed25519.pub
ssh -T git@github.com

# Good response means auth works:
# Hi USERNAME! You have successfully authenticated, but GitHub does not provide shell
access.
```

5. Push, Pull and Fetch

5.1 git push

What: `git push` uploads commits from your local repository to GitHub. It does not upload unsaved working directory changes.

```
git add .
git commit -m "Update README documentation"
git push origin main

# first push of a feature branch
git push -u origin feature/update-readme
```

Situation	Wrong thinking	Correct action
After editing file	Run git push immediately	First git add, git commit, then git push
No commits yet	Push should upload files	Push uploads commits only
Working on feature	Push directly to main	Push feature branch and create PR
Authentication error	Keep trying password	Use PAT for HTTPS or SSH key for SSH

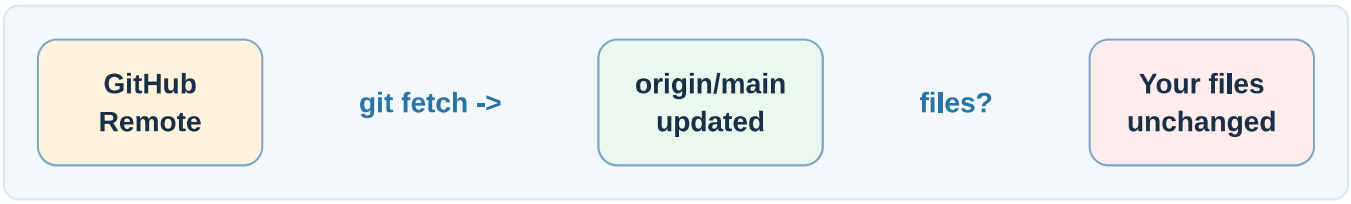
5.2 git pull

What: `git pull` downloads changes from GitHub and merges them into your current branch.

```
git checkout main
git pull origin main
```

Common mistake	Problem	Solution
Pull with uncommitted changes	May conflict or block pull	Commit, stash, or discard local changes first
Pull into wrong branch	Changes go into unexpected branch	Run git branch, then switch correctly
Pull without checking	Files change suddenly	Use git fetch first if you want to inspect

5.3 git fetch



`git fetch` downloads remote information, but it does not change your working files. It is safer when you want to inspect what changed on GitHub before merging it.

```
git fetch origin
git log --oneline main..origin/main
git diff main..origin/main
# when ready
git pull origin main
```

Command	Meaning	Changes files?
git fetch	Download remote information only	No
git pull	git fetch + merge	Yes

> **Interview line:** git pull = git fetch + git merge.

6. Branches, Merge, Pull Request and GitFlow

6.1 Branches

A branch is a separate line of work. It lets you develop a feature, fix a bug, or update documentation without disturbing stable main code.

```
git branch
git checkout -b feature/add-login
git checkout main
git branch -d feature/add-login

# modern alternative
git switch -c feature/add-login
git switch main
```

Purpose	Professional branch name
New feature	feature/login-page
Bug fix	bugfix/fix-502-error
Terraform change	feature/add-vnet-module
Documentation update	docs/update-readme
Hotfix	hotfix/fix-prod-port

> **Professional rule:** Do not work directly on main for normal feature work. Create a feature branch first.

6.2 Merge

Merge combines changes from one branch into another branch. In companies, merging into main usually happens through a Pull Request instead of direct manual merge.

```
git checkout main
git merge feature/add-login
```

A merge conflict happens when Git cannot automatically decide which version is correct because the same lines were changed differently.

```
<<<<<<< HEAD
your current branch change
=======
incoming branch change
>>>>>>> feature/add-login
```

Conflict solution steps
Open the conflicted file.
Choose the final correct content.
Remove conflict markers: <<<<<<<, =======, >>>>>>>.
Save the file.
Run git add .
Run git commit or continue the merge if Git asks.

6.3 Pull Request

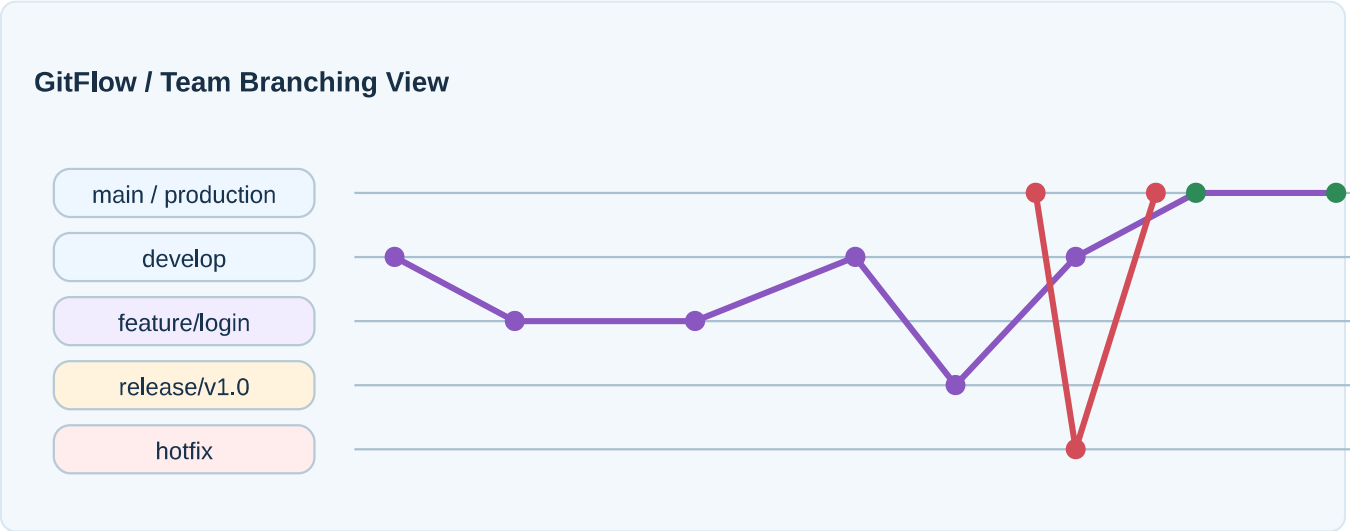
A Pull Request (PR) is a request to review and merge your branch into another branch, usually feature branch into main. PRs create a professional review gate.

```
git checkout main
git pull origin main
git checkout -b feature/update-readme
# edit files
git add .
git commit -m "Update README documentation"
git push -u origin feature/update-readme
# GitHub -> Create Pull Request -> Review -> Merge
```

Wrong way	Why wrong	Correct way
Push every change directly to main	No review, risky	Push feature branch and create PR
Huge PR with unrelated changes	Hard to review	Create small focused PRs
Commit message: update stuff	Unclear history	Use clear messages

Merge without testing	Can break app	Run local tests before PR

6.4 GitFlow architecture



GitFlow separates stable production code, integration work, new features, release preparation, and urgent hotfixes. Beginners do not need every detail immediately, but must understand why branches exist: safety, review, and clean history.

7. Undo and Recovery



Situation	Safe command	Explanation
Changed file, not committed	git restore file	Discard local file changes
Staged by mistake	git restore --staged file	Unstage without deleting file
Need temporary save	git stash	Store dirty work temporarily
Bad shared commit	git revert COMMIT_ID	Create rollback commit safely
Lost commit	git reflog	Find previous HEAD movements
Need deleted branch back	git checkout -b recovered COMMIT_ID	Recover work into a new branch

Important: git reset --hard removes uncommitted changes from your working tree. Do not use it casually. In teams, prefer git revert for shared branch rollback.

7.1 Stash practical use

Use stash when you have unfinished local changes but need to quickly switch branches or pull latest changes.

```
git status
git stash
git pull origin main
git stash pop
```

Mistake	Fix
Using stash and forgetting it	Run git stash list
stash pop creates conflict	Resolve conflict, then add/commit
Stashing important long-term work	Better create a branch and commit it

8. Secure Repository Hygiene

Professional DevOps engineers protect credentials. Do not commit passwords, private keys, PAT tokens, Terraform tfstate files containing secrets, .env files, or cloud credentials.

```
# Good .gitignore example
.env
*.pem
*.key
*.pfx
*.tfstate
*.tfstate.backup
.terraform/
__pycache__/
*.log
node_modules/
.vscode/
```

If secret was committed	Action
Private repo	Immediately revoke or rotate the secret
Public repo	Treat the secret as compromised
Already pushed	Do not only delete in next commit; remove from history if required and rotate secret
Terraform state	Use secure remote backend and restricted access

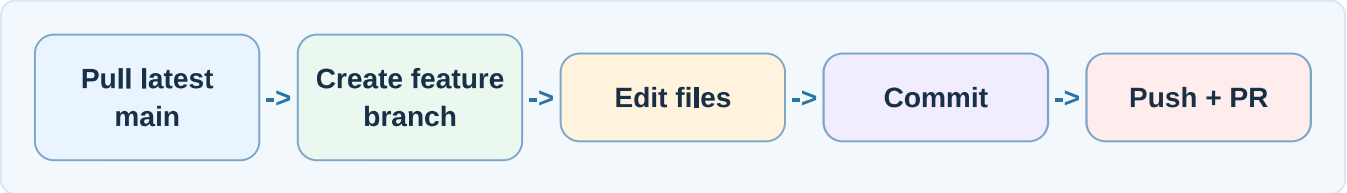
> **Golden rule:** .gitignore prevents future tracking. It does not automatically remove files already committed. For already tracked files, use `git rm --cached file`, commit, and rotate exposed secrets if needed.

9. Linux, PowerShell, MobaXterm and Windows Notes

Task	Linux / Git Bash	PowerShell
Current folder	pwd	pwd
List files	ls -la	dir
Create folder	mkdir project	mkdir project
Create file	touch app.py	New-Item app.py
Delete file	rm notes.txt	del notes.txt
Print SSH public key	cat ~/.ssh/id_ed25519.pub	cat $env:USERPROFILE\.ssh\id_ed25519.pub

Git concepts are the same in Linux, MobaXterm, Git Bash, and PowerShell. The main differences are shell commands, file paths, and authentication flow.

10. Complete Professional Workflow




```
git checkout main
git pull origin main

git checkout -b feature/my-change
# edit/create/delete files
git status
git diff
git add .
git diff --staged
git commit -m "Add clear professional message"
git push -u origin feature/my-change

# GitHub: Create Pull Request -> Review -> Merge

git checkout main
git pull origin main
git branch -d feature/my-change
```

Step	Why it matters
git checkout main	Start from stable branch
git pull origin main	Avoid working on old code
git checkout -b feature/my-change	Create safe working branch
git diff	Review unstaged changes

git add .	Stage selected changes
git diff --staged	Review exactly what will be committed
git commit	Save local checkpoint
git push	Upload branch to GitHub
Pull Request	Review and merge safely
Delete branch	Keep repo clean

11. Scenario-Based Hands-on Labs

Lab 1: Create local repo and push to GitHub

Goal: Create a repo locally, track it with Git, connect it to GitHub, and push professionally.

```
mkdir git-real-practice
cd git-real-practice
git init
echo "Git practice project" > README.md
git add README.md
git commit -m "Add README"
git branch -M main
git remote add origin <your-github-repo-url>
git push -u origin main
```

> You learned: commit must exist before push; origin is the remote; -u sets upstream for future pushes.

Lab 2: Feature branch and PR

```
git checkout main
git pull origin main
git checkout -b feature/add-second-line
echo "Second line" >> README.md
git diff
git add README.md
git diff --staged
git commit -m "Add second line to README"
git push -u origin feature/add-second-line
```

Open GitHub, click Compare & pull request, add title/description, create PR, merge it, then pull main locally.

Lab 3: Fetch before pull

```
git checkout main
git fetch origin
git log --oneline main..origin/main
git diff main..origin/main
git pull origin main
```

You learned: fetch is safer for inspection; pull changes your files because it fetches and merges.

Lab 4: Merge conflict practice

```
git checkout main
echo "version from main" > conflict.txt
git add conflict.txt
git commit -m "Add conflict file from main"
git checkout -b feature/conflict-test
echo "version from feature" > conflict.txt
git add conflict.txt
git commit -m "Update conflict file from feature"
git checkout main
echo "version changed in main" > conflict.txt
git add conflict.txt
git commit -m "Update conflict file from main"
git merge feature/conflict-test
```

Open conflict.txt, choose final content, remove markers, run `git add conflict.txt`, then commit.

Lab 5: Undo before commit

```
echo "bad line" >> README.md
git status
git diff
git restore README.md
git status
```

Lab 6: Unstage a file

```
echo "temporary" > temp.txt
git add temp.txt
git status
git restore --staged temp.txt
git status
```

Lab 7: Revert a bad commit

```
git log --oneline
git revert <commit-id>
git log --oneline
```

Lab 8: Secure .gitignore

```
echo ".env" > .gitignore
echo "*.tfstate" >> .gitignore
echo ".terraform/" >> .gitignore
git add .gitignore
git commit -m "Add Git ignore rules"
```

12. Common Mistakes, Why They Happen, and Fixes

Mistake	Why it happens	Fix

git status shows nothing after commit	Status shows current pending changes, not history	Use git show or git log -1
git push says up-to-date but file missing	File was not committed	git status, git add, git commit, git push
Deleted file appears in status	Deletion not committed yet	git add deleted-file or git rm, then commit
Accidentally worked on main	Forgot branch	Create branch immediately: git checkout -b feature/name
Pull causes conflict	Local and remote changed same lines	Resolve file, git add, commit
Wrong remote URL	origin points to wrong repo	git remote -v then git remote set-url origin URL
New branch push fails	No upstream branch	git push -u origin branch-name
Too many files staged	Used git add . blindly	git restore --staged file
Password auth fails	GitHub does not accept account password for Git operations	Use PAT or SSH
Secret committed	Sensitive file entered history	Rotate secret; remove from repo/history if needed

13. Interview Questions and Scenario Answers

Question	Strong answer
What is Git?	Git is a distributed version control system used to track and save code changes so teams can manage history and collaborate.
What is GitHub?	GitHub is a remote hosting platform for Git repositories, collaboration, PRs, issues, and code review.
What is staging?	Staging is the preparation area where selected changes are placed before commit.
What is commit?	A commit is a saved snapshot of staged changes in the current branch history.
What is push?	Push uploads local commits to a remote repository.
What is pull?	Pull downloads remote changes and merges them into the current branch.
What is fetch?	Fetch downloads remote information without changing working files.
Fetch vs pull?	Fetch is safe inspection; pull is fetch plus merge.
Why use branches?	Branches isolate work and protect main from unreviewed or unstable changes.
What is PR?	A Pull Request is a review request to merge one branch into another.
What is merge conflict?	It happens when Git cannot automatically combine changes to the same lines.
Safe rollback on shared branch?	Use git revert, not reset --hard, because revert preserves shared history.

Scenario interview examples

Scenario	Professional answer

Production broke after a commit	Identify the bad commit using logs, revert it with git revert, push the revert commit, then investigate root cause in a feature branch.
Pushed branch but PR not showing	Check branch exists on remote, compare branch manually, verify base branch, and ensure changes are committed.
Need to switch branch but have unfinished work	Commit to a temporary branch or use git stash, then switch branch.
File missing on GitHub after push	It was likely not committed. Run git status, git add, git commit, then push.
Accidentally committed secret	Revoke/rotate secret immediately; remove from repository and history if needed.

14. DevOps / CI-CD Concept View



You do not need to master CI/CD or Docker before Git. But understand the connection: a push to GitHub can start automation. That automation can build, test, scan, package, and deploy your application or infrastructure.

Stage	Beginner meaning	Learn now?
Git push	Developer uploads commits	Yes
CI trigger	Pipeline starts automatically	Concept now
Build/Test	Automation checks code	Concept now
Docker image	Application packaging	Later
Azure deploy	Release to cloud	Later

15. Professional README Template

Use this structure in portfolio projects so your GitHub repository looks like real cloud/DevOps documentation.

```
# Project Title

## Overview
Short explanation of what the project does.

## Architecture / Workflow
Add architecture image or workflow PNG.

## Tools Used
Git, GitHub, Linux, Azure, Terraform, etc.

## Implementation Steps
1. Created project structure
2. Created local Git repository
```

```
3. Created branch
4. Pushed code
5. Opened PR

## Issues Faced and Fixes
- Authentication failed: fixed using SSH/PAT
- Merge conflict: resolved manually

## What I Learned
Short professional learning summary.
```

Weak commit message	Professional commit message
update	Update README with architecture workflow
fix	Fix backend API connection error handling
changes	Add Terraform module outputs
final	Finalize documentation and screenshots

16. 7-Day Premium Git Practice Plan

Day	Task	Outcome
1	Install Git and configure identity	Ready machine
2	Create repo, README, commit, push	Lifecycle understood
3	Create feature branch and push	Branch confidence
4	Open PR and merge	Team workflow
5	Create and resolve conflict	Conflict fear removed
6	Practice restore, revert and stash	Recovery confidence
7	Create portfolio README with diagrams	Documentation skill

> **Success criteria:** you can explain the lifecycle without notes, push using SSH or PAT, work with branches and PRs, and recover from common mistakes safely.

17. Final Command Cheat Sheet

Need	Command
Check status	git status
See unstaged changes	git diff
Stage file	git add filename
Stage all	git add .
Review staged changes	git diff --staged
Commit	git commit -m "message"

Show latest commit details	git show
Show last commit only	git log -1
Short history	git log --oneline
Create branch	git checkout -b feature/name
Switch branch	git checkout main
Push branch	git push -u origin feature/name
Pull latest main	git pull origin main
Fetch remote info	git fetch origin
Merge branch	git merge feature/name
Delete local branch	git branch -d feature/name
See remotes	git remote -v
Change remote URL	git remote set-url origin URL
Undo file change	git restore file
Unstage file	git restore --staged file
Temporary save	git stash
Recover lost work	git reflog

> **Final professional rule:** Change -> Review -> Stage -> Commit locally -> Push branch -> Pull Request -> Merge -> Pull latest main. This is the real Git workflow used in cloud and DevOps projects.

18. References and Source Note

This master guide was built by combining the two uploaded PDFs: the visual Git for DevOps documentation and the professional Git workflow guide, then expanding them with additional step-by-step explanations, scenarios, mistakes, solutions, secure repository hygiene, and interview preparation.

Recommended public references for continued learning: Official Git documentation, GitHub Docs, GitHub SSH documentation, GitHub PAT documentation, and beginner Git cheat sheets from reputable learning sites.

> This document is a learning and portfolio-preparation guide. Commands should be tested in a safe practice repository before applying to production repositories.